Les Graphes : séance 4

Prérequis :

- **Avoir fait les séances précédentes sur les graphes : vocabulaires, matrice d'adjacence et dictionnaire d'adjacence, classes Graphe_or et Graphe**
- **le concept de récursivité**

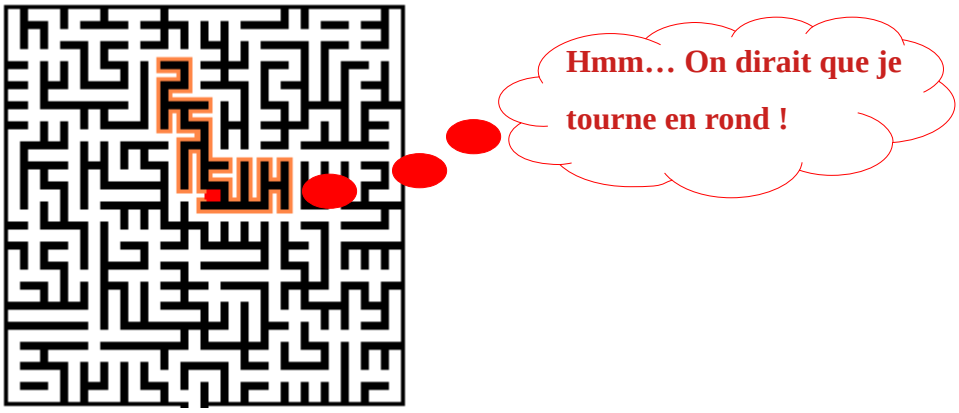
Objectifs :

- **savoir dérouler à la main l'algorithme du parcours en profondeur sur un graphe donné**
- **comprendre les algorithmes de parcours en profondeur (itératif et récursif) et de recherche d'un chemin entre deux sommets, puis savoir les implémenter en Python à partir des pseudo-codes**

Partie VI : parcours en profondeur d'un graphe – DFS

On a coutume de dire que pour sortir d'un labyrinthe il suffit de toujours suivre le mur situé sur notre droite. En particulier, on emprunte toujours la première porte que l'on rencontre sur notre droite et, si on arrive à un cul de sac, cela a pour effet de nous faire faire un demi-tour.

Malheureusement, cette méthode peut échouer dans certains cas, où on se retrouve à tourner en rond autour d'un bloc.



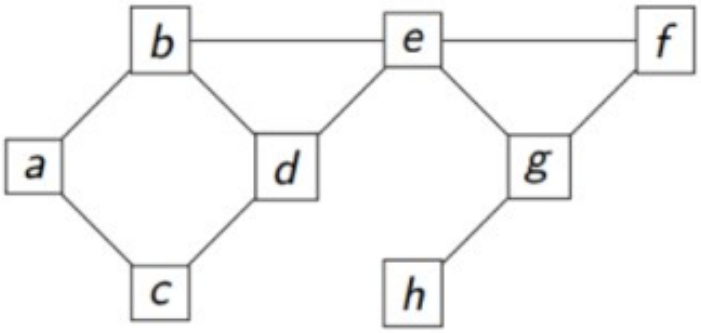
Pour y remédier, on peut marquer les endroits par lesquels on est déjà passé. Quand on revient sur un embranchement qui est marqué, on procède comme s'il s'agissait d'un cul de sac. On peut aussi emprunter les embranchements de façon arbitraire : aller à gauche ou à droite. Ces deux idées combinées constituent un algorithme fondamental appelé **parcours en profondeur.**

Pour plus de détail : https://fr.wikipedia.org/wiki/Algorithme_de_parcours_en_profondeur

Pour un graphe, on commence en un sommet, on passe à un de ses voisins, et ainsi de suite. Autrement dit, on commence à aller le plus profond possible. S'il n'y a pas de voisin, le programme revient au sommet précédent et passe à un autre de ses voisins.

La méthode – DFS (Depth First Search)

Parcourir un graphe en profondeur à partir d'un sommet, consiste à explorer le graphe en suivant un chemin.
Lorsqu'on arrive sur un sommet qui n'a plus de voisins non visités, on le marque.
Puis on remonte dans le chemin pour explorer les voisins non visités d'un autre sommet…
On utilise une pile et deux listes. Prenons en exemple ce graphe :



On dispose d'un graphe(G), de deux listes (sommets_visités, sommets_fermés) et d'une pile(p).

Méthode

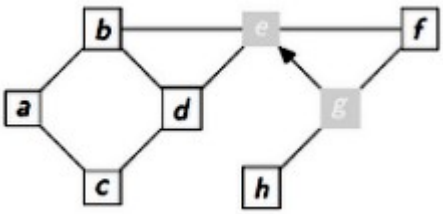
♦ Le sommet de départ est par exemple 'g', on l'empile.

♦ On met le sommet de départ dans la liste sommets_visités.

♦ Puis tant que la pile n'est pas vide :

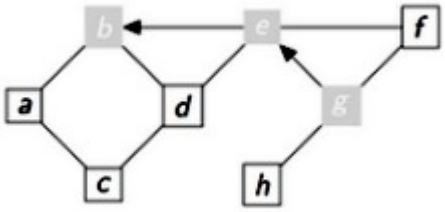
- On récupère le sommet de la pile dans une variable *tmp*
- voisins reçoit la liste des voisins de *tmp* non déjà visités
- Si voisins n'est pas vide :
  - v ← un voisin choisi au hasard
  - sommets_visités ← v
  - On empile v
- Sinon :
  - sommets_fermés ← *tmp*
  - On dépile p

Voici les contenus des variables au premier tour de la boucle tant que :



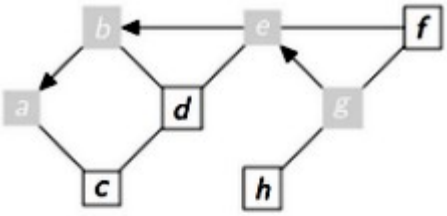
1er tour
tmp : g
voisins : ['e', 'f', 'h']
v: e
sommets_visités : ['g', 'e']
pile : ['g', 'e']
sommets_fermés : []

Voici les contenus des variables au second tour de la boucle tant que :

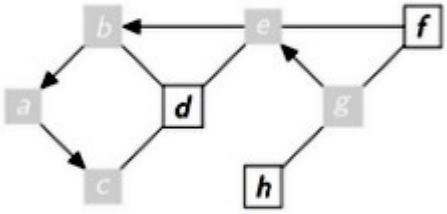


2e tour
tmp : e
voisins : ’b’, ’d’, ’f’]
v: b
sommets_visités : ’g’, ’e’, ’b’]
pile : ’g’, ’e’, ’b’]
sommets_fermés : []

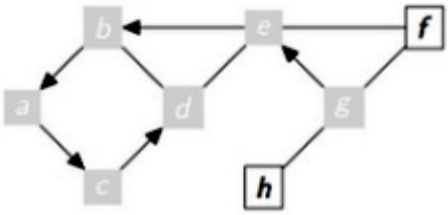
Pour la suite, compléter les contenus des variables dans le cadre ci-dessous :



3e tour
tmp : b
voisins : [a,d]
v: a
sommets_visités :[g,e,b,a]
pile : [g,e,b,a]
sommets_fermés : [c

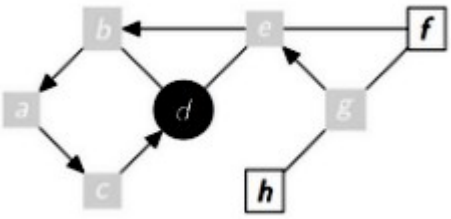


4e tour
tmp : a
voisins : c
v: c
sommets_visités : g,e,b,a,c
pile : g,e,b,a,c,
sommets_fermés :[]



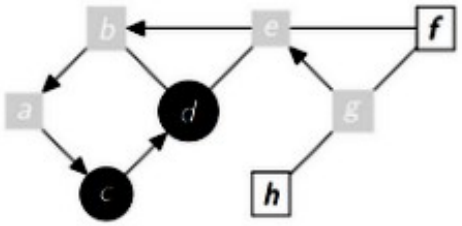
5e tour
tmp :c
voisins :d
v:d
sommets_visités : g,e,b,a,c,d,
pile :g,e,b,a,c,d,

Voilà le contenu des variables au sixième tour de la boucle tant que :

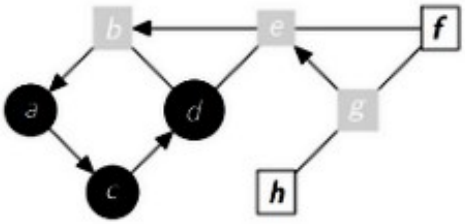


6e tour
tmp : d
voisins : []
v: d
sommets_visités : ’g’, ’e’, ’b’, ’a’, ’c’, ’d’]
pile : ’g’, ’e’, ’b’, ’a’, ’c’]
sommets_fermés : ’d’]

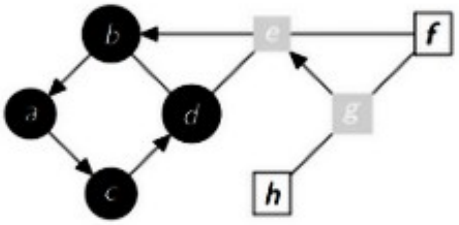
Travail à poursuivre, encore 9 tours !



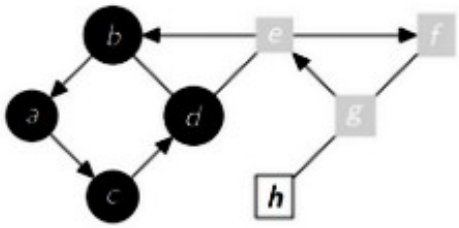
7e tour
tmp : c
voisins :
v: c
sommets_visités :
pile : ['g','e','b','a']
sommets_fermés : d,c,



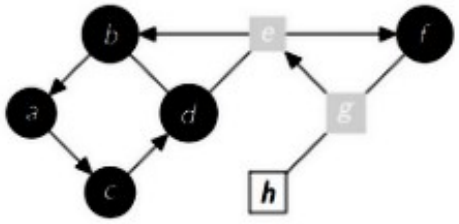
8e tour
tmp : a
voisins :
v: a
sommets_visités :
pile : ['g','e','b']
sommets_fermés : ['d','c','a']



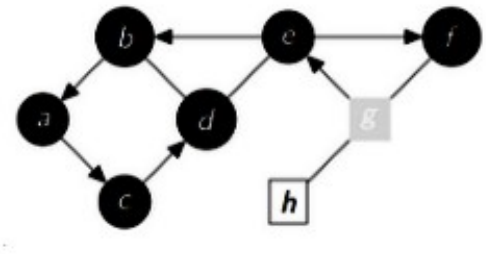
9e tour
tmp : b
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



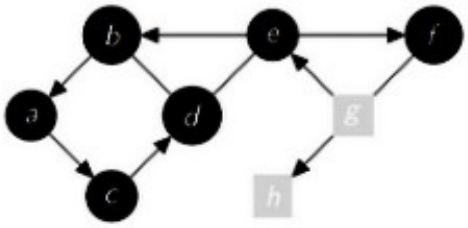
10e tour
tmp :
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



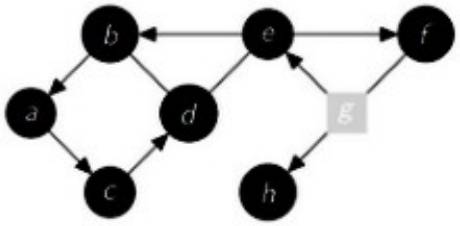
11e tour
tmp :
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



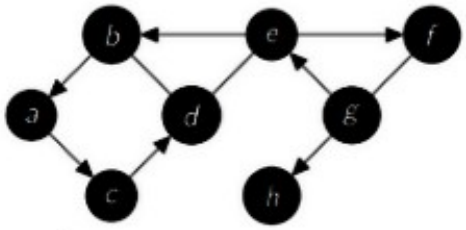
12e tour
tmp :
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



13e tour
tmp :
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



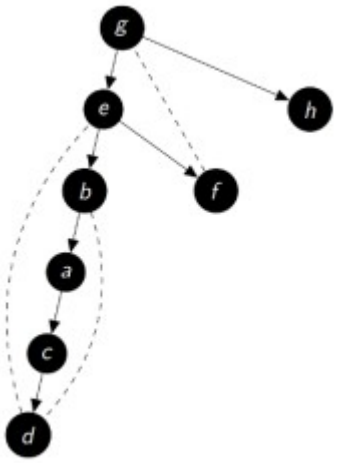
14e tour
tmp :
voisins :
v:
sommets_visités :
pile :
sommets_fermés :



15e et dernier tour
tmp :
voisins :
v:
sommets_visités :
pile :

Remarque : Comme les choix dans la liste des voisins sont aléatoires, il y a plusieurs parcours possibles.

Au final l'arborescence associée au parcours en profondeur peut donc être modélisée de la façon suivante :



Implémentation du DFS en Python

Exercice 1

1. Écris le code en Python pour la création du dictionnaire qui représente le graphe G précédent représenté page 2. Enregistre ce programme sous *C38-DFS_iteratif.py.*

2. Tu as besoin pour la suite d'utiliser une pile :
   - Enregistre le programme Pile.py (qui est dans l'espace des classes) dans le même dossier que ton programme *C38-DFS_iteratif.py.*
   - Importe la classe Pile ainsi : *from Pile import **

3. Complète le programme en implémentant l'algorithme ci-dessous en Python et teste-le sur le graphe G avec le·s sommet·s de ton choix comme sommet·s de départ.

L'algorithme itératif du DFS en pseudo-code

```
fonction dfs_iteratif(G,s):
    # G est le graphe et s le sommet de départ
    sommets_visités ← []
    p ← Pile()
    sommets_visités ← sommet
    On empile s dans p
    Tant que p n'est pas vide faire
        tmp ← le sommet de la pile
        voisins ← la liste des voisins de tmp non déjà visités
        Si voisins n'est pas vide alors
            v ← un voisin au hasard
            sommets_visités ← v
            On empile v
        Sinon
            on dépile p
    fin tant que
    renvoyer sommets_visités
```

Remarques : - comme les choix dans la liste des voisins sont aléatoires, il y a plusieurs parcours possibles.
- c'est une version sans utiliser la liste *sommets_fermés*

Indication 1 : la bibliothèque *random* permet un choix aléatoire dans une liste :

import random
v=random.choice(voisins)

Indication 2 : voici une ligne de code qui permet de récupérer les voisins de tmp non déjà visités :

voisins=[x for x in G[tmp] if x not in sommets_visites]

DFS – Version récursive

On peut utiliser un algorithme récursif pour parcourir un graphe en profondeur.
En voici la description :

1. On part d'un sommet du graphe.
2. On le marque comme visité s'il ne l'est pas déjà.
3. Pour chacun de ses voisins non visités, on reprend à partir du 1.

Il y a une « boucle » du 3. au 1. Cela présage une méthode récursive.

L'algorithme récursif du DFS en pseudo-code

```
fonction dfs_recursif(G,s,visite=[]):
    # G est le graphe, s le sommet de départ, visite la liste des sommets visités
    Si s n'est pas dans la liste visite alors
        ajouter s dans la liste visite
        voisins ← la liste des voisins de s qui ne sont pas dans la liste visite
        Pour chaque voisin dans voisins faire
            dfs_recursif(G,voisin,visite)
    renvoyer la liste visite
```

Exercice 2 :

1. Code l'algorithme de la **fonction *dfs_recursif*** *en Python* et enregistre ton programme sous ***C38-dfs_recursif.py.***
2. Teste-le avec notre graphe G et, par exemple, avec 'g' comme sommet de départ.

   Le résultat attendu est : ['g', 'e', 'b', 'a', 'c', 'd', 'f', 'h'].

Application : un chemin entre deux sommets

L'objectif est de faire afficher un chemin entre deux sommets d'un graphe.

Par exemple : a-b-e-g-h est l'un des chemins possible entre a et h pour notre graphe G.

La méthode consiste à mémoriser les « sommets voisins » du sommet visité comme clés d'un dictionnaire et ayant pour valeur son parent (le sommet visité).

Le sommet de départ n'aura bien entendu pas de parent (None).

On reprend le code de l'algorithme récursif en ajoutant un paramètre qui est le dictionnaire *parents.*
Si *s* est le sommet de départ, on l'initialisera ainsi : *parents={s:None}*

L'algorithme récursif du DFS modifié en pseudo-code

```
fonction dfs_recursif2(G,s,parents,visite=[]):
    Si s n'est pas dans la liste visite alors
        ajouter s dans la liste visite
        voisins ← la liste des voisins de s qui ne sont pas dans la liste visite
        Pour chaque voisin dans voisins faire
            parents[voisin]=s
            dfs_recursif2(G,voisin,parents,visite)
    renvoyer le dictionnaire parents
```

Cette fonction renvoie un dictionnaire qui contient les sommets visités(clés) et leurs parents(valeurs).

Il faut maintenant exploiter ce dictionnaire pour faire afficher un chemin entre deux sommets.

C'est ce que réalise cette fonction, codée en Python, qui prend en paramètre l'arrivée et le dictionnaire parents :

```
def solution(arrivée, parents):
    chemin = []
    courant = arrivée
    while courant != None:
        chemin = [courant] + chemin
        courant = parents[courant]
    return chemin
```

Exercice 3 :

1. Enregistre le **programme *C38-chemin_dfs.py*** (qui est dans l'espace des classes) dans ton dossier.
2. Code l'algorithme de la **fonction *dfs_recursif2*** en Python dans le **programme *C38-chemin_dfs.py***.
3. Teste-le pour faire afficher un chemin entre les sommets a et h de notre graphe G.